

How ResumeIQ Works

A working reference for the backend and frontend: what each layer is responsible for, how a résumé becomes a ranked candidate, where state lives, and which parts are load-bearing. Written against the code as it stands on **Aasrith-New-phase-4**.

Layer	Stack	Entry point
Backend	FastAPI, Pydantic v2, Python 3.11	backend/app/main.py
Frontend	React 19, TanStack Router, Vite, Tailwind v4	frontend/src/routes/__root.tsx
Storage	Azure Blob Storage (JSON documents)	backend/app/services/azure_services.py
Search	Qdrant + Azure AI Search (dual)	backend/app/services/vector_store.py
AI	Azure OpenAI gpt-5, text-embedding-3-small	backend/app/services/azure_services.py
Tests	417 backend (pytest), 9 frontend (vitest)	backend/tests, frontend/src/**

There is no SQL database. Every entity is a JSON blob addressed by a key, and the repository layer is what makes that behave like a database. Understanding that one decision explains most of the rest of the backend.

Part 1 — Backend

The request path

Every API call runs the same four steps. Routes stay thin on purpose: they resolve who is calling, validate input through Pydantic, delegate to a service, and return a response model. Business logic lives in services so it can be tested without HTTP.

```
HTTP request
-> dependencies.py resolve identity + role (get_current_user)
-> routes/*.py validate payload, authorise
-> services/*.py the actual work
-> storage/repository read/write JSON documents
-> Azure Blob Storage
```

Identity and authorisation

Identity comes from an opaque session token in an `Authorization: Bearer` header. Tokens are random 32-byte URL-safe strings stored on the user document, so signing out revokes them server-side — a signed stateless token could not be revoked. Passwords are PBKDF2-HMAC-SHA256, 240,000 iterations, with a per-user salt.

This is recent and important. Until it was fixed, every data endpoint trusted an `X-Recruiter-Email` header that had a default value. Any caller could read any recruiter's candidates by changing a string, with no token at all. The header now only works when `ALLOW_HEADER_IDENTITY` is on, which exists for the test suite and local scripts.

```
# A valid token wins over any header claim.
# An invalid token is a 401 -- it never falls back to the header,
# or an expired session would silently become whoever the header names.
ALLOW_HEADER_IDENTITY=false # <- set this in any deployment
```

Roles

Three roles only: **recruiter**, **hiring_manager**, **it_admin**. HR is deliberately absent — recruiters and hiring managers own hiring decisions and IT owns the platform, so an HR role would carry no permission the other three do not already hold. `normalise_role()` maps anything arriving from a form or an identity provider onto one of the three, falling back to the least-privileged rather than raising.

Gate	Who	What it protects
CanRunPipeline	recruiter, it_admin	Create jobs, upload résumés, add employees
CanDecide	all three	Advance / select / hire / reject a candidate
IsAdmin	it_admin	Platform administration

A refusal returns 403 naming your role and the allowed set, rather than a bare denial.

Storage: documents, not tables

JsonBlobStore is the only thing that talks to Azure Blob Storage. It provides `get` / `put` / `delete` / `exists` / `list_prefix` plus batched `get_many` / `put_many` / `delete_many`, an ETag-validated cache, and a connection pool sized to the batch concurrency (azure-core defaults to 10 connections, which throttled every batched read).

`put_many(strict=True)` propagates the first failure — for primary domain writes that must not be silently lost. The default logs and continues, which is what secondary writes like evidence and history want.

Repository shapes

The key encodes the access pattern, which is why there are several repository types rather than one:

Repository	Key shape	Why
Repository	collection/{id}.json	Plain entity, addressed by id
AppendOnlyRepository	collection/{sortable}.json	Log; never updated in place
PartitionedAppendOnly	collection/{partition}/{sortable}.json	History per candidate
EvaluationRepository	evaluations/{job}/{candidate}.json	Key IS the uniqueness constraint
EvidenceRepository	evidence/{evaluation}.json	One doc per evaluation, not per row

Two consequences worth knowing. Evaluations are upsert-by-overwrite — the key encodes (job, candidate), so re-scoring replaces rather than duplicates. And three collections cannot be deleted by their own id, which is why the reset script clears them by prefix instead.

The résumé intake pipeline

This is the core flow of the product. Uploads are processed in a background task so the request returns immediately; the UI polls each upload's status.

```
POST /api/resumes/upload (files + job_id + source)
status=queued row written, bytes stored in blob storage
status=ocr 40% Document Intelligence extracts text
.txt/.md are decoded directly -- sending them
to OCR fails
status=parsing 70% gpt-5 returns structured fields
status=complete 100% Candidate upserted (dedupe by email),
then embedded and indexed for search
```

job_id and source are required at intake. A candidate with no role has nowhere to appear, and internal employees and external applicants are different populations with different workflows. Both are chosen before the dropzone accepts a file.

Who belongs to a job

Board membership has exactly three routes in, and having an evaluation is deliberately not one of them:

- The résumé was uploaded against this role (`candidate.job_id`)
- A recruiter placed them here explicitly (a `CandidatePlacement`)
- They are in the general pool (no role) *and* have been scored here

Membership used to be "has an evaluation for this job". Ranking a job scores the whole pool, so one ranking run wrote an evaluation for every candidate against that job and put everyone on every board.

`rankable_candidate_ids_for_job()` is a wider set than `candidate_ids_for_job()` for exactly this reason: being scored is how a pool candidate joins a board, so ranking must include them even though display must not.

Scoring

Two independent scoring systems, and they answer different questions.

Weighted match score

`candidate_matcher.rank_candidates()` scores each candidate per category and combines them with a `WeightConfig`: skills 40%, experience 30%, education 15%, certifications 10%, projects 5%, communication 10%. The recruiter's saved weights are sent from the frontend, so the server and the UI agree on the number.

ATS benchmark (dual scoring)

A deliberately dumb keyword scan — literal matches of required and nice-to-have skills, required weighted double — run beside an LLM semantic read. The interesting signal is the *delta*: a large positive delta means a keyword-only ATS would have filtered out someone worth interviewing.

Both halves are nullable, which matters. A job with no skills listed has **no keyword baseline** — it used to score 100, handing every candidate a flawless result off zero matched keywords. And a failed semantic call reports nothing rather than copying the keyword score, which made the delta exactly 0.00 and the verdict "aligned": agreement between one real score and a copy of itself.

Vector search

Candidates are embedded (text-embedding-3-small, 1536 dims) from a document built out of their name, title, skills and experience — contact details are excluded because they dilute the vector. `VECTOR_BACKEND` selects the engine:

Value	Behaviour
blob	Vectors in the document store, exact cosine. No extra service.
search	Azure AI Search vector index (HNSW, cosine).
qdrant	Qdrant, embedded or server. Purpose-built ANN.
dual	Qdrant for vectors, Azure for keywords, fused with RRF.

Two details that are easy to get wrong. Scoping is pushed into the engine's own filter, not applied afterwards — otherwise the service returns other recruiters' rows for us to discard. And scores are normalised: Azure reports $1/(2 - \text{cosine})$ for a cosine profile, so it is converted back before it reaches the UI, which renders it as a percentage. A fused (hybrid) result has *no* comparable similarity and reports null rather than inventing

one.

Every backend returns None — not an empty list — when it cannot be reached, so "could not ask" stays distinguishable from "no matches" and the caller falls back to the local index.

Hiring progress

`hiring_progress.py` computes, per candidate, a checklist of the same steps in the order they happen, plus **the single next action**. That calculation is on the server so no two views can disagree about what to do next.

```
screened -> assessment -> interview rounds -> decision

assessment states: todo | current (sent) | done | skipped
next_action.kind: send_assessment | wait | advance | decide
next_action.goto: which workspace tab performs it
```

The assessment step is three-valued on purpose. A board that only records completion makes "nobody has looked at this" and "we decided this person does not need one" identical, and that is exactly what an inexperienced recruiter cannot reconstruct later. Skipping records a reason and who decided.

Hiring outcome

`candidate.hiring_status` is **active | hired | rejected** and is the one fact every other view reads. Before it existed, "hired" lived only in a decisions log and a stage derived from it, so the ranking list happily offered Hire on someone already hired. Hiring someone also converts them: source becomes internal, `employment_status` becomes assigned, and their assignment is set to the role title.

Email

SMTP over STARTTLS. The critical property is honesty: with no credentials configured the service returns `sent=False`, `source="mock"` and an explanatory error. It previously returned `sent=True`, so records, audit logs and the UI all claimed a candidate had been emailed when nothing left the machine.

The test suite is blocked from sending. Once real credentials landed, every pytest run delivered genuine mail to the fixtures' example.com addresses.

Part 2 — Frontend

Shape

File-based routing under `src/routes`, one file per screen. `__root.tsx` decides everything above the page:

```
QueryClientProvider
ThemeProvider
public route? -> Outlet only (no shell, no app state)
otherwise -> RequireAuth
AppStateProvider
CopilotProvider
AppShell -> Outlet
```

Public routes are `/welcome`, `/login`, `/register`. Everything else renders inside the shell and assumes a session exists.

The API layer

`lib/api.ts` is the single boundary to the backend — 97 exported functions, one per endpoint. Every call goes through `recruiterHeaders()`, which attaches the session token and the recruiter email. In-flight GETs are deduplicated by URL + recruiter, so independent components asking for the same thing at the same moment produce one request.

Client state

State	Lives in	Notes
Session	localStorage + <code>/api/auth/me</code>	Token is the authority
activeJobId	AppStateProvider (memory)	Candidates are ranked against it
blindMode	AppStateProvider	Defaults ON; hides names
Scoring weights	localStorage, per account	Saved explicitly; sent to server
Preferences	localStorage, keyed by email	Signature and defaults only

Identity is deliberately *not* in localStorage any more. It was merged over the session, so a value saved once won permanently — including after signing in as somebody else, which is how one account displayed another recruiter's name on outgoing candidate emails.

The screens that matter

Route	What it owns
/	Dashboard: counts, recent roles, the three-step explainer
/jobs/\$jobId	The job workspace. Hiring steps, candidates, board, JD, upload
/candidates	Ranked list for the active job, weights panel, filters
/internal-hiring	Internal roles and the people who can fill them
/external-hiring	External roles and the applicant funnel
/bench	Internal roster; who is available and who is assigned
/actions	Notification inbox; refreshes on an interval and on focus
/settings	Account identity, company context, vector index status

The job workspace

The most important screen. It opens on **Hiring steps** — the checklist — rather than a table of scores, because that is the view that tells someone what to do next. The board is where people are moved between rounds, and the checklist links to it when that is the next action.

Moving a card records a stage. It does not email anyone by itself, but the success toast offers to, and taking that offer sends a real message.

Design system

Everything is driven by CSS custom properties in `styles.css` — `--primary`, `--card`, `--border` and so on, in `oklch`. Because every component reads those tokens, changing the palette restyles the whole app without touching a route file.

- **Type:** Syne for display headings, Space Grotesk for UI. Syne has deep descenders, so display headings carry a line-height floor.
- **Motion:** one easing curve throughout; all of it disabled under `prefers-reduced-motion`.
- **Layout:** `.flow-grid` / `.flow-stack` use `clamp()` so spacing scales continuously rather than stepping at breakpoints.

One trap worth remembering: `background-clip: text` with `color: transparent` paints only inside the element's box, so on an inline element descenders and trailing punctuation fall outside it and render as nothing. Vertical padding on the inline box fixes it without disturbing line layout.

Part 3 — Running and operating it

Local development

```
npm run dev:all # both servers; reclaims its own stale ports
frontend http://localhost:8080
backend http://localhost:8000 (docs at /docs)
```

```
cd backend && .venv/bin/python -m pytest -q # 417 tests
cd frontend && npx tsc --noEmit && npx vitest run
```

scripts/dev.sh only reclaims a port when the process holding it is one of ours and its working directory is inside the checkout. Anything else it refuses and tells you what to kill.

Configuration

Variable	Purpose
AZURE_STORAGE_CONNECTION_STRING	Blob storage. Or ACCOUNT_NAME + ACCOUNT_KEY.
AZURE_OPENAI_API_KEY / ENDPOINT	gpt-5 for parsing, scoring, questions
AZURE_OPENAI_EMBEDDING_*	text-embedding-3-small, 1536 dims
AZURE_SEARCH_ENDPOINT / ADMIN_KEY	Vector + keyword index
VECTOR_BACKEND	blob search qdrant dual
SMTP_USERNAME / SMTP_PASSWORD	Outbound email. Gmail needs an app password.
ALLOW_HEADER_IDENTITY	MUST be false in any deployment
ASSESSMENT_BASE_URL	Currently a placeholder host

Scripts

- `backup_and_reset.py` — snapshots every document to JSON before clearing, and keeps user accounts unless told otherwise. Restores with `--restore FILE`.
- `compact_blob_documents.py` — consolidates legacy per-row documents.
- `assign_legacy_owner.py` — claims records written before candidates had an owner.

Known gaps

Read this before demoing or deploying.

Gap	Consequence
ALLOW_HEADER_IDENTITY defaults true	Auth fix is inert until set false
ASSESSMENT_BASE_URL is a placeholder	Assessments can never complete; Skip is the way past
No Entra ID integration	Password auth only
CandidateMatcher.get_candidate_score x2	First definition is dead code, pre-existing
/people route is unlinked	Reachable by URL only
Frontend tests cover 2 components	Harness exists; coverage is thin

A note on reading this document: everything above describes behaviour verified against the running system, not intent. Where something is known to be incomplete it is in the table immediately above rather than described as if it worked.